

Introduction to Machine Learning

Student Lesson Guide & Interactive Code Worksheet | Scikit-Learn Edition

1. What is Machine Learning?

Imagine sorting delicious Indian snacks like **Jalebi** and **Dosa**. Both look different, taste different, and have unique shapes! Humans instantly recognize which is which because we have seen many examples over time.

Machine Learning (ML) is the science of teaching computers to learn directly from examples and data, rather than following rigid, predefined rules. Just like students learn by practicing problems, machines discover underlying patterns on their own!

2. Traditional Programming vs. Machine Learning

Traditional Programming	Machine Learning
How it works: Programmers write strict, explicit rules and instructions for the computer to execute. Example: The rules of an IPL cricket match are fixed. A score algorithm adds runs based on set rules. Limitation: The computer cannot adapt if something new happens that wasn't coded in advance.	How it works: The computer is fed historical data (examples) and discovers the rules or patterns automatically. Example: Sorting Alphonso mangoes by weight, softness, and color without writing manual rules. Advantage: The system improves and adapts as it sees more data.

3. Core Building Blocks: Features (X) & Labels (y)

When a machine looks at a dataset, it separates the information into two critical components:

Features (X) — The Clues

Features are the input variables or properties that describe the situation. They provide the clues the machine uses to learn.

Cricket Example: Overs remaining, Runs required, Wickets in hand.

Label (y) — The Answer

The label is the target outcome or prediction value that the machine is trying to learn and predict.

Cricket Example: Final Match Outcome (Win or Lose).

4. Supervised Learning & Practical Example

Supervised Learning is like learning with a teacher or a cricket coach. The teacher provides the training data containing both the clues (Features X) and the correct answers (Labels y). The machine studies these pairs to learn the relationship.

Real-World Application: Sorting Alphonso Mangoes

A fruit vendor wants to classify mangoes automatically as **Ripe** or **Raw**.

- **Features (X):** Colour Score (how yellow/orange) & Softness Level.
- **Label (y):** Ripe or Raw.

The supervised ML model learns the exact color threshold and softness combination that distinguishes a sweet ripe Alphonso from a raw one!

5. The Scikit-Learn 3-Step Recipe

In Python, the `scikit-learn` library makes training a machine learning model as easy as following a 3-step recipe:

- 1 **Import & Instantiate Model:** Choose an algorithm (e.g., `DecisionTreeClassifier`, `LinearRegression`) and create the model object.
- 2 **Train Model (Fit):** Call `model.fit(X, y)` so the machine learns patterns from the features and labels.
- 3 **Make Predictions:** Pass new feature data into `model.predict(X_new)` to get predictions!

6. Step-by-Step Practical Code Examples

Example 1: Classification — Predicting Cricket Team Selection

Predicting player selection based on Practice Hours and Batting Average using a Decision Tree Classifier.

```
from sklearn.tree import DecisionTreeClassifier

# Step 1: Prepare Training Data (Features X: [Practice Hours, Batting Avg])
X = [[5, 30], [3, 15], [8, 45], [2, 10]]
y = ['Selected', 'Not Selected', 'Selected', 'Not Selected']

# Step 2: Initialize and Fit the Model
model = DecisionTreeClassifier()
model.fit(X, y)

# Step 3: Predict for a new player (6 practice hours, 35 batting avg)
new_player = [[6, 35]]
prediction = model.predict(new_player)
print("Selection Prediction:", prediction[0]) # Output: Selected
```

Example 2: Classification — Sorting Alphonso Mangoes (Ripe vs. Raw)

Categorizing fruit based on Color Score (1-10) and Softness Score (1-10) using a K-Nearest Neighbors (KNN) model.

```

from sklearn.neighbors import KNeighborsClassifier

# Features X: [Color Score (1-10), Softness Score (1-10)]
X = [[8, 7], [2, 3], [9, 9], [3, 2], [7, 8]]
y = ['Ripe', 'Raw', 'Ripe', 'Raw', 'Ripe']

# Step 1 & 2: Instantiate and Train KNN Model (3 Neighbors)
mango_classifier = KNeighborsClassifier(n_neighbors=3)
mango_classifier.fit(X, y)

# Step 3: Test a new batch of mangoes
new_mangoes = [[8, 6], [2, 4]]
predictions = mango_classifier.predict(new_mangoes)
print("Mango Predictions:", predictions) # Output: ['Ripe', 'Raw']

```

Example 3: Regression — Predicting Exam Scores Based on Study Hours

Predicting continuous numeric values (Exam Score) instead of categories using Linear Regression.

```

from sklearn.linear_model import LinearRegression

# Features X: [Study Hours per day, Practice Tests taken]
X = [[2, 1], [4, 2], [6, 4], [8, 5], [10, 7]]
y = [55, 68, 80, 91, 98] # Target continuous output: Exam Score (%)

# Step 1 & 2: Instantiate and Fit Linear Regression
regressor = LinearRegression()
regressor.fit(X, y)

# Step 3: Predict score for student studying 5 hrs/day with 3 practice tests
student_data = [[5, 3]]
predicted_score = regressor.predict(student_data)
print("Predicted Score:", round(predicted_score[0], 2)) # Output: ~74.5

```

Example 4: Data Splitting & Evaluation — Training vs. Testing Sets

In real-world ML, we split data into training and test sets to evaluate model performance on unseen data.

```

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.tree import DecisionTreeClassifier

# Larger Dataset: [Study Hours, Attendance %] -> Pass (1) / Fail (0)
X = [[1, 50], [2, 60], [3, 70], [4, 65], [6, 80], [7, 85], [8, 90], [9, 95]]
y = [0, 0, 0, 0, 1, 1, 1, 1]

# Split data: 75% for Training, 25% for Testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Train model
clf = DecisionTreeClassifier()
clf.fit(X_train, y_train)

# Evaluate accuracy on test set
y_pred = clf.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Model Accuracy:", accuracy * 100, "%")

```

7. Model Evaluation, Bias, and Data Quality

Machine learning models are only as good as the data used to train them. Mistakes and poor predictions happen when training data is biased, incomplete, or corrupted.

Garbage In, Garbage Out!

If we train a cricket prediction model using data only from right-handed batters, the machine may fail or make unfair decisions when evaluating left-handed players.

- **Biased Data:** Leads to unfair and skewed predictions.
- **Missing / Bad Data:** Causes inaccurate model performance.

Key Takeaway: Always ensure your dataset is balanced, clean, and representative!

Take-Home Mission!

Scenario: Build a mental ML model to predict whether it will rain during cricket practice tomorrow in Mumbai!

Identify what features you would collect (e.g., humidity, cloud cover, wind speed) and what label you want to predict (Rain / No Rain).

STUDENT ASSESSMENT & PRACTICE WORKSHEET

Test your understanding of Machine Learning concepts, Scikit-Learn workflows, code implementations, and data ethics.

Question 1: Conceptual Understanding

Explain the key difference between Traditional Programming and Machine Learning using a real-world example other than cricket.

Question 2: Features vs. Labels

In an ML application designed to predict whether a student will pass an exam, identify at least two possible Features (X) and the target Label (y).

Question 3: Scikit-Learn Syntax

What is the purpose of the `.fit(X, y)` method in Scikit-Learn, and how does it differ from `.predict(X_new)`?

Question 4: Code Analysis & Completion

Suppose you have a dataset with features $X = [[4, 20], [7, 40]]$ representing practice hours and score, and labels $y = ['Retry', 'Pass']$.

Write down the full Python code required to train a `DecisionTreeClassifier` on this data and predict the outcome for a student with $[6, 35]$.

Question 5: Practical Hands-On Coding

Write a complete Python snippet using `LinearRegression` from `sklearn.linear_model` to train on house sizes $X = [[500], [1000], [1500]]$ and prices $y = [30, 60, 90]$ (in Lakhs), then predict the price of a 1200 sq. ft. house.

Question 6: Data & Ethics

What does the phrase "Garbage In, Garbage Out" mean in Machine Learning? Give an example of how biased training data can lead to unfair decisions.
